

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (12/05)

1. Consider the following 32-bit word (in hexadecimal): 0x3F900000. What is the **single-precision IEEE 754 floating-point** value encoded in this word? Recall that the standard uses an 8-bit exponent with a bias of 127. Specify your answer in decimal (and show your work).

Answer:

sign = 0

exponent = 127 – bias = 0

significand = 1.001₂

Thus $1.001_2 \times 2^0 = 1.125$

2. You are asked to design a new architecture for machine learning on the edge. The architecture should be extremely low power. You propose to use a **custom normalized floating-point representation** that has less exponent and fraction bits than the IEEE 754 standard. After profiling your machine learning applications, you find that you only need to represent **unsigned numbers** within the range of **0.12890625** to **19.25**.
 - a) First, you consider an **unsigned fixed-point** encoding that uses the **minimum number of bits** needed to represent both 0.12890625 and 19.25. Specify each of the following (and show your work):
 - i. Number of **integer** bits (to the left of the binary point)
 - ii. Number of **fractional** bits (to the right of the binary point)

Answer:

0.12890625 → 0.00100001

19.25 → 10011.01

- i. **Number of integer bits = 5**
- ii. **Number of fractional bits = 8**

- b) Next, design a **custom floating-point** representation that requires the **minimum number of bits** to represent both 0.12890625 and 19.25. Your representation should have the following simplified format:

Exp (with Bias)	Fraction (with implicit leading 1)
-----------------	------------------------------------

You do not need a sign bit nor support for any special values (e.g., zero, infinity, NaN, denormalized). With your representation, the value is simply computed as:

$$1.\text{Fraction} \times 2^{(\text{Exp} - \text{Bias})}$$

Specify each of the following (and show your work):

- i. Number of bits for **Fraction** (excluding the leading 1)
- ii. Number of bits for **Exp**
- iii. Value of **Bias** (in decimal)

Answer:

$$\begin{array}{lll} 0.12890625 & \rightarrow 1.00001 \times 2^{-3} & \rightarrow 1.00001 \times 2^{(0-3)} \\ 19.25 & \rightarrow 1.001101 \times 2^4 & \rightarrow 1.001101 \times 2^{(7-3)} \end{array}$$

- i. Number of bits for **Fraction** = 6
- ii. Number of bits for **Exp** = 3
- iii. Value of **Bias** = 3

- c) You implement your **custom floating-point** representation from Q2b above but forget to support the **implicit leading 1**. It now encodes the following instead:

$$0.\text{Fraction} \times 2^{(\text{Exp} - \text{Bias})}$$

Because of this bug, it is now possible to encode some numbers in **multiple** ways using this representation. List **all** the possible encodings of the number 0.375 (decimal) with your buggy custom floating-point format (i.e., same Bias and number of Fraction and Exp bits as in Q2b). Provide your answer in binary.

Answer:

$$0.375_{10} \rightarrow 1.1_2 \times 2^{-2}$$

Thus:

- 010 110000
- 011 011000
- 100 001100
- 101 000110
- 110 000011